Краткая архитектура (mutual-TLS)

- **Raspberry Pi + Zymkey** — **TLS-сервер** (API `/sign`) с приватным ключом, хранящимся внутри Zymkey; Pi отвечает на запросы подписи только от аутентифицированных клиентов.
- **Media server (Ubuntu + TPM2.0)** — **TLS-клиент**, хранит клиентский приватный ключ в TPM и использует его для client-certificate authentication при соединении с Pi.
- Канал — **mutual-TLS** с проверкой сертификатов и CRL/OCSP; все операции логируются локально и в зашифрованный журнал.

Почему так (ключевые преимущества)

- **Приватные ключи никогда не покидают защищённый модуль**: серверный ключ — в Zymkey; клиентский ключ — в TPM.
- **Аппаратный root of trust** на обеих сторонах (TPM + Zymkey) даёт сильную гарантию, что подписи выполняются на доверенных устройствах.
- **Air-gap не обязателен**, но канал защищён и аутентифицирован; при необходимости можно комбинировать с физическим переносом.

Конкретный развёртывательный план (пошагово)

1) Подготовка CA и политики

- Создать **локальный CA** (offline) или использовать существующий; определить срок действия сертификатов и политику ротации.
- Выпустить **CA-root** и **intermediate**; хранить root в сейфе/HSM (escrow).

2) Сертификаты и ключи

- **На Raspberry Pi (сервер):**
  - Сгенерировать ключ **внутри Zymkey** через Zymbit API; получить CSR (если Zymkey поддерживает CSR) или сгенерировать публичный ключ и сформировать CSR на Pi.
  - Подписать CSR у CA → получить серверный cert. Серверный приватный ключ остаётся в Zymkey и **не экспортируется**.
- **На media server (клиент):**
  - Сгенерировать клиентский приватный ключ **в TPM** (через `tpm2_create`/`tpm2_pkcs11`), создать CSR, подписать у CA → установить клиентский cert в систему.
  - Настроить `tpm2-pkcs11` или OpenSSL engine для использования TPM-ключа в TLS-стеке.

3) Настройка TLS сервера на Pi

- Развернуть signing-daemon, который:
  - слушает HTTPS (nginx/stunnel/embedded HTTPS) с **server cert** и использует Zymkey для приватного ключа;
  - требует **client certificate** и проверяет цепочку до доверенного CA;
  - проверяет CN/SAN и дополнительные атрибуты (например, device ID или attestation token).
- Пример: nginx с `ssl_client_certificate` и `ssl_verify_client on`; приватный ключ реализуется через PKCS#11 engine или Zymbit-wrapper.

4) Настройка TLS клиента на media server

- Установить `tpm2-pkcs11` и зарегистрировать TPM-ключ в PKCS#11-слое.
- Настроить клиент (curl/openssl/собственный скрипт) использовать PKCS#11 URI для TLS client auth.
- Пример OpenSSL client команд:

```
# пример с pkcs11 engine (псевдокод)
openssl s_client -connect pi:443 -cert client.crt -engine pkcs11 -keyform
engine -key "pkcs11:object=client;type=private"
```

5) Протокол подписи (API)

- **POST /sign** (HTTPS, mutual-TLS)
  - Body: `{ "manifest_hash": "<sha256>", "metadata": {...} }`
  - Server: проверяет client cert, проверяет права (ACL), вызывает `Zymkey.sign(manifest_hash)`, возвращает `{ "signature": "<base64>", "pubkey": "<pem>" }`.
  - Лог: запись операции (who, when, manifest id, hash, result) в неизменяемый журнал.

6) Логирование и аудит

- Логи операций подписи пишутся локально в `/var/log/signing.log` и периодически архивируются в зашифрованный контейнер (например, GPG с ключом в HSM/escrow).
- Включить tamper-лог Zymkey и системный журнал TPM-операций; экспортировать события в SIEM при возможности.

Команды и инструменты (примерный набор)

- **TPM**: `tpm2_create`, `tpm2_createprimary`, `tpm2_pkcs11` (для интеграции с PKCS#11).
- **Zymkey**: Zymbit SDK (Python/C) — `zk.sign(data)`; использовать systemd-daemon для API.
- **TLS**: nginx/stunnel/openssl; настроить `ssl_verify_client on` и `ssl_client_certificate /etc/ca/ca-chain.pem`.
- **PKCS#11**: `p11tool`, `pkcs11-tool` для проверки ключей в TPM.

- **Verification**: `openssl dgst -sha256 -verify pub.pem -signature sig.bin manifest.xml`.

Безопасность и операционные требования (обязательные)

- **MFA и RBAC** для доступа к медиасерверу и Pi; SSH только через jump-host.
- **Физическая защита Pi**: запираемый корпус, tamper-пломбы, журнал доступа.
- **Процедуры обновления**: только подписанные прошивки; обновления по change-control.
- **Escrow**: резервный зашифрованный бэкап публичных и (если возможно) резервных ключей хранить в HSM/сейфе; процедура восстановления — M-of-N.
- **Time**: синхронизировать RTC Zymkey перед сессией; логировать время подписи.

Приёмочные тесты (обязательные)

- **Mutual-TLS handshake**: успешное соединение только с клиентским cert из TPM; отказ для других.
- **Sign/verify**: подпись реального DCP-манифеста и верификация на медиасервере.
- **ACL**: попытка подписи от неавторизованного клиента — отклонение.
- **Tamper**: симуляция вскрытия корпуса Pi — проверка реакции и блокировки ключей.
- **Recovery**: восстановление ключа/сертификата из escrow и успешная подпись.